PyRCH: Algorithms and Solvers for Min-Max Constrained Heterogeneous Multi-Robot Routing Problems

Yiyu Wang[1], Yunxiang Yao[1], Shizhe Zhao[1], Zhongqiang Ren[1†]

***Abstract*— Similar to the well-known multiple traveling salesman problem (mTSP) and vehicle routing problem (VRP), this paper considers min-max routing problems with constraints that seek a set of paths, one for each robot, to visit each node in a graph exactly once, while minimizing the length of the longest path (min-max). The robots are heterogeneous in a sense that nodes are subject to assignment constraints (AC): each node can only be visited by a subset of robots. Besides AC, the problem also includes time window constraints and resource constraints. Such routing problems naturally arise in autonomous exploration of unknown environments and logistics of warehouse and factories. The contribution of this paper includes both adaption of our prior work to address these routing problems, and a public solver `PyRCH`[1]. While several existing solvers, such as Google OR-Tool, PyVRP, and LKH, are available, they are mainly designed for min-sum optimization and they rarely consider heterogeneity among the robots such as the AC. This work focuses on min-max optimization and heterogeneous robots, which complements these existing solvers. Results show that our solver often runs faster and finds similar or better solution quality than existing OR-Tool and PyVRP solvers in the presence of AC and other constraints.**

I. Introduction

Given a graph, Traveling Salesman Problem (TSP) seeks a path starting from the depot to visit each node in the graph exactly once and return to the depot, while minimizing the path length [1]. Multiple Traveling Salesman Problem (mTSP) generalizes TSP by finding a set of paths, one for each robot, so that each node in the graph is visited exactly once by any path [2]–[4], while minimizing the sum or the maximum of path costs, which are referred to as min-sum or min-max mTSP respectively. Besides, a closely related problem is the Vehicle Routing Problem (VRP) [5] that further considers the capacity constraints of the robots. TSP is NP-hard [1] and so are mTSP and VRP, with either min-sum or min-max costs.

Although mTSP and VRP have been researched a lot, few of them consider heterogeneity among the robots, which often arises as node-robot assignment constraints (AC) where only a subset of robots are eligible to visit each node. Such heterogeneity naturally arises in robotic systems, and we are mainly motivated by autonomous exploration and logistics. First, in autonomous exploration, consider that a team of heterogeneous robots, such as drones, legged, and wheeled robots, are tasked to autonomously map a bounded unknown environment as quick as possible [6], [7], which may include various terrains such as stairs, rivers, and flat ground. During exploration, the boundaries between known and unknown spaces are often referred to as the frontiers and the robots need to quickly and iteratively solve mTSP to determine the allocation and visit orders of frontiers so that the robots can map the environment quickly. As the robots move, new areas are mapped and the frontiers are updated, where a new mTSP needs to be solved again. In this case, the nodes are subject to ACs since only a subset of robots

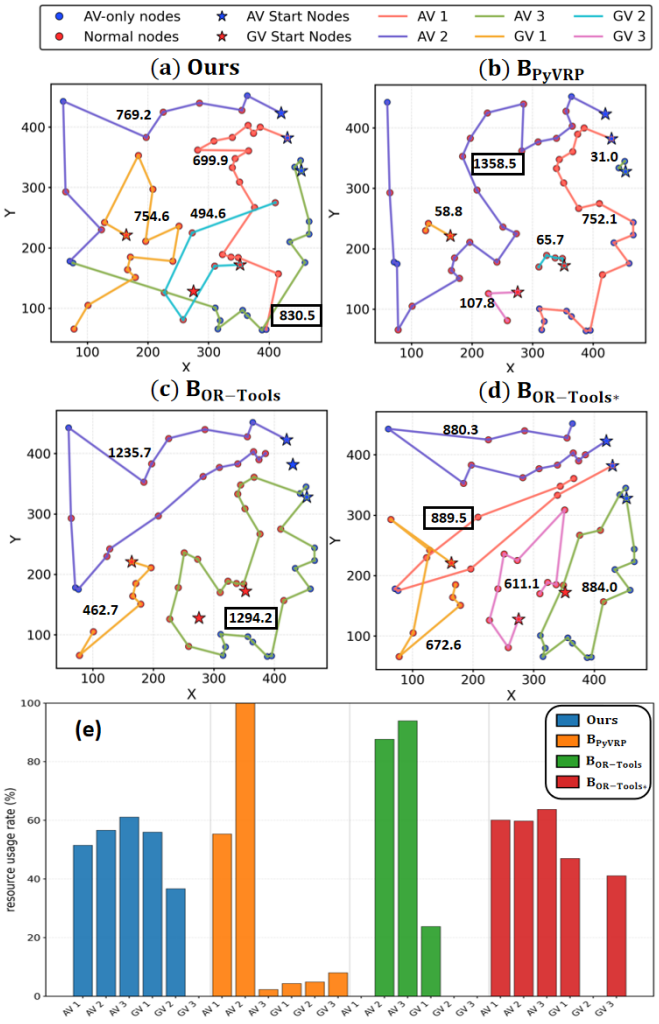


Fig. 1: Representative solutions for RCH with AC and RC. (a)–(d) show the paths generated by our solver and baselines. Boxed numbers indicate the max path length. (e) shows the resource (e.g. battery) usage (%) per robot. AV is aerial vehicle and GV is ground vehicle.

[1]The authors are at Global College, Shanghai Jiao Tong University in China. (Correspondence: zhongqiang.ren@sjtu.edu.cn) This work was supported by the Natural Science Foundation of China under Grant 62403313.

[1]`https://pypi.org/project/PyRCH/`

can access certain frontiers due to their ability to traverse different terrains. Second, in logistics [8], [9], robots often need to service a collection of machines or storage for item transportation, loading or unloading of machines. Here, AC arises from different payload or manipulation capabilities of the agents. Time window constraints (TW) help enforce the required service time ranges, and resource constraints (RC) correspond to battery capacity or payload limits.

With that in mind, this paper considers min-max Routing with Constraints for Heterogeneous robots (RCH). The RCH studied in this work includes three types of hard constraints: AC, TW, and RC. The goal of this paper is to (i) develop algorithms to address these RCH problems, and (ii) encapsulate our algorithms into a public solver for this family of routing problems for easy usage by the community.

To address mTSPs, a variety of advanced algorithms were developed, but most of them are not open sourced and are not easy to implement for beginners in this area. Among the public solvers that can solve TSP and VRP, the most popular ones include Google OR-Tools [10], PyVRP [5] and LKH [11]. However, we find that they are mainly designed for min-sum optimization, and their performance degrades when tuned to handle min-max optimization. Besides, they rarely consider heterogeneity among the robots such as the AC in the problem, which can reduce their performance obviously. This work focuses on min-max optimization and heterogeneous robots, which complements these existing solvers. Our algorithm is based on A*-like search for routing problems [12], [13] and is closely related to the algorithm in our prior work HEHA [7]. While HEHA [7] focuses solely on AC and autonomous exploration, this work further considers RC and TW with new pruning rules and heuristics, and develops a public solver for these routing problems.

For verification, we test our solver against Google OR-Tools and PyVRP on various routing problems with mixtures of AC, RC and TW constraints. Our solver consistently achieves lower min-max objective than the baselines on problem of AC or AC combined with RC. For problems with TW, our solver finds solutions with quality comparable to PyVRP and better than OR-Tools. In addition, our solver is anytime and can find high-quality solutions faster than the baselines while providing solution sub-optimality bounds.

II. Related Work

Routing problems have been approached by exact [1], [14], [15], bounded sub-optimal approximation [2], [3], unbounded sub-optimal heuristics [11], and learning [16], [17], trading off solution quality guarantees for runtime efficiency. Among the existing routing algorithms, most of them consider min-sum optimization [2], [3], as opposed to min-max. In addition, few of them consider assignment constraints [3], [4]. In particular, transformation methods [8], [14] can handle assignment constraints, but they can only minimize the sum of costs, which can lead to long makespan and increase the exploration time. For public solvers such as the popular Google OR-Tools [10] and PyVRP [5], although some of them provide options and parameters to configure the solver to indirectly optimize min-max objectives subject to AC, their algorithms are not designed for min-max routing with AC, RC, and TW, which can limit their performance.

The RCH considered in this paper arises in exploration, logistics and possibly other applications. In [7], heterogeneous robots are tasked to explore unknown environments which requires solving RCH quickly and iteratively during the exploration of new areas. In logistics, several algorithms [8], [18] were developed to plan collision-free paths of the robots to visit multiple target locations in cluttered warehouse like environments. These algorithms often rely on a fast mTSP solver as a building block and then further considers robot-robot collision avoidance constraints.

III. Problem Definition

Let $G = (V, E)$ denote a graph, where V is the set of N_v target nodes, and E is the set of edges connecting pairs of nodes. Let $I = \{1, 2, \cdots, N_r\}$ denote a set of N_r robots. We use a superscript i over a variable to indicate to denote robot-specific quantities. The travel-time cost between two nodes $u, v \in V$ of robot i is denoted as $c^i(u, v)$. Each robot i seeks a path $\pi^i = \{v^i_1, v^i_2, \cdots, v^i_\ell\}$, an ordered list of nodes in G, starting at the depot $v^i_s \in V$ and terminating at a goal depot node $v^i_g \in V \cup \{null\}$. When $v^i_g = null$, the robot path can end at any node in G.

Each target node $v \in V$ must be visited exactly once by a single robot, subject to node-robot AC: each node v is associated with a set of robots $A(v) \subseteq I$ that is capable of visiting v. Each node $v \in V$ is associated with a TW $[t_a(v), t_b(v)] \in \mathbb{R}^+ \times \mathbb{R}^+$. Let $t^i(v^i_j)$ denote the arrival time at node v^i_j along a robot i's path π^i, which is the sum of edge costs up to v^i_j in π^i, i.e., $t^i(v^i_j) = \sum_{l=1}^{j-1} c^i(v^i_l, v^i_{l+1})$, and must satisfy the TW constraint $t_a(v^i_j) \leq t^i(v^i_j) \leq t_b(v^i_j)$. If a robot i arrives early at a node (i.e., $t^i(v^i_j) < t_a(v^i_j)$), the robot can wait till $t^a(v^i_j)$, which means $t^i(v^i_{l+1}) = \max\{t_a(v^i_{l+1}), t^i(v^i_l) + c^i(v^i_l, v^i_{l+1})\}$. Each robot $i \in I$ is subject to a RC, such as the battery limit, which is represented as a travel time limit $T^i > 0$. Let $C(\pi^i)$ denote the cost of path π^i, which is the arrival time of the last node in π^i, and must satisfy $C(\pi^i) \leq T^i$.

Let $J(\pi) = \max_i C(\pi^i)$ denote the *makespan* (i.e., the maximum individual path cost), which is the objective function to be minimized. The RCH problem seeks a joint path π that satisfies all constraints while minimizing $J(\pi)$.

IV. Method

A. Label Representation

Our approach follows the general A* search framework [19] by iteratively expanding paths from the start towards the goal while pruning unpromising ones. We first present our algorithm and then discuss its difference from the existing algorithm in our prior work HEHA [7].

1) Label and Partial Solution: A label $l = (\vec{v}, \vec{t}, B)$ represents a partial solution over all robots, where

- $\vec{v} = (v^1, v^2, \cdots, v^{N_r})$ is a vector representing the current nodes of robots,

- $\vec{t} = (t^1, t^2, \cdots, t^{N_r})$ denotes the cumulative travel costs of the robots,
- $B \subseteq V$ is the set of visited nodes.

Let $\vec{v}(l)$, $\vec{t}(l)$, $B(l)$ denote the corresponding components.

The initial label is $l_0 = (\vec{v}_s, \vec{0}, \emptyset)$, where $\vec{v}_s = (v_s^1, \ldots, v_s^{N_r})$. A label is associated with a unique partial solution and can be reconstructed by backtracking its parent pointers to l_0. A goal label corresponds to a complete solution (i.e., a joint path that all targets in V). Depending on whether v_g^i is $null$ or not as aforementioned, a goal label may additionally require each robot to end at its designated goal depot v_g^i.

2) Bounds and Dominance: The solver maintains an incumbent solution cost UB (upper bound) from the best complete solution found so far. For a label l, the algorithm uses a value $f(l)$ as an estimated lower bound on the cost of any complete solution by extending l. The computation of $f(l)$ depends on the selected objective and heuristic as explained later. To reduce redundant search, we introduce the following dominance rule to compare and prune labels, which is similar to our prior work that uses A*-like search to solve TSP or HPP problems [12], [13].

Definition 1 (Label Dominance)**.** Given two labels $l_1 = (\vec{v}_1, \vec{t}_1, B_1)$ and $l_2 = (\vec{v}_2, \vec{t}_2, B_2)$, we say that l_1 *dominates* l_2, denoted $l_1 \prec l_2$, if the following conditions hold: (i) $\vec{v}_1 = \vec{v}_2$; (ii) $\vec{t}_1$ is element-wise no greater than $\vec{t}_2$, which is denoted as $\vec{t}_1 \leq \vec{t}_2$; (iii) $B_2 \subseteq B_1$. We also write $l_1 \preceq l_2$ if $l_1 \prec l_2$ and $l_1 = l_2$.

If label $l_1 \preceq l_2$, then l_2 cannot lead to a better solution path since any future path from l_2 can be cut-and-paste to l_1 without worsening the objective.

3) Focal List: The solver uses focal search to speed up the computation while ensuring bounded sub-optimal solution. Focal search maintains an open list L_O and a focal list L_F. L_O contains all labels to be expanded, where labels are prioritized based on their f-values from the minimum to the maximum. L_F is a subset of L_O containing labels with f-value within $[f_{\min}, (1+\epsilon)f_{\min}]$, where $f_{\min}$ is the smallest f-value of labels in L_O.

L_F prioritizes labels based on the following three values in lexicographic order:

- (i) the number of visited nodes $|B(l)|$,
- (ii) the f-value $f(l)$, and
- (iii) $g_{\text{sum}}(l) := \sum_{i \in I} g^i(l)$ the total path cost of all robots in label l.

In other words, for any two labels l_1, l_2, l_1 has higher priority than l_2 in L_F, if (i) $|B(l_1)| > |B(l_2)|$, or (ii) $|B(l_1)| = |B(l_2)|$ and $f(l_1) < f(l_2)$, or (iii) $|B(l_1)| = |B(l_2)|$, $f(l_1) = f(l_2)$ and $g_{\text{sum}}(l_1) < g_{\text{sum}}(l_2)$. If all three values are equal, ties are broken arbitrarily. The search selects the label with the highest priority in L_F for expansion. Focal prefers earlier expansion of labels that have visited more nodes.

B. MST-based Heuristics

For a label l, let $g_{\max}(l) = \max_{i \in \mathcal{I}} t^i(l)$ denote the current makespan. Let $U(l) = V \setminus B(l)$ be the set of unvisited nodes. We estimate the remaining cost-to-go summed over all robots by a minimum spanning tree (MST) based lower bound heuristic $h(l)$ and then compute the averaged cost-to-go as $\frac{h(l)}{N_f}$, where N_f is the number of "free" robots that have not reached their goals yet. Let I_f denote the set of "free" robots. To build this MST, all current nodes of the free robots are connected with zero-cost edges and then a MST rooted on any one of those current nodes is built to span all unvisited nodes. Then, we define $f_{\text{est}}(l) = \frac{h(l)}{N_f} + \min_{i \in I_f} t^i(l)$, which underestimates the makespan of any possible solution by extending l. Then, the f-value used in search is $f(l) = \max\{g_{\max}(l),\, f_{\text{est}}(l)\}$.

When there is no TW, this heuristic $h(l)$ is same as the one proposed in HEHA [7]. When there are TWs, the computation of MST and $h(l)$ differs from HEHA [7], and we use a TW-aware MST. Specifically, the cost of each edge (u, v) is computed by considering the bi-directional traversal and the time windows. When a robot i moves from u to v, a lower bound traversal cost is $c^i_{\text{lb}}(u, v) = \max\big(c^i(u, v),\, t_a(v) - t_b(u)\big)$, where the second term considers the unavoidable waiting time at u due to the time windows if the robot arrives at v too early. Besides, if robot i cannot reach v within the required time window $[t_a(v), t_b(v)]$ with any departure time in $[t_a(u), t_b(u)]$, we set $c^i_{\text{lb}}(u, v) = \infty$. We compute both $c^i_{\text{lb}}(u, v)$ and $c^i_{\text{lb}}(v, u)$ and set the edge cost as $w_{\text{TW}}(u, v) = \min_{i \in \mathcal{I}} \min\big(c^i_{\text{lb}}(u, v), c^i_{\text{lb}}(v, u)\big)$ when computing the MST, whose cost is then used as $h(l)$.

Algorithm 1: PyRCH

```
Initialize l_0 and compute f(l_0) ← Eval(l_0)
UB ← +∞; π* ← ∅
Add l_0 to L_O and L_F
while within time limit do
    while L_F ≠ ∅ do
        l ← L_F.pop()
        Remove l from L_O
        if IsDom(l) or f(l) ≥ UB then
            continue
        UpdateAndFilter(l)
        if IsComplete(l) then
            π ← PostOptimize(l; P)
            π* ← π; UB ← Cost(π)
            break
        foreach l' ∈ SeqExpand(l) do
            if NotFeasible(l'; P) or IsDom(l') then
                continue
            parent(l') ← l
            f(l') ← Eval(l')
            Add l' into L_O and update L_F
        UpdateFocal(L_O, L_F, ε)
    Decrease ε
return π*
```

C. Anytime Focal Search

As shown in Alg. 1, the focal search adds the initial label l_0 into L_O and L_F and starts the search. In the inner loop, the label l with the highest priority in L_F is selected for

expansion and removed from L_O and L_F. Before expanding l, we check whether it is dominated by any existing label in $L(\vec{v}(l))$, which is the subset in L_O and L_F containing all labels with same robots' positions as $\vec{v}(l)$. If l is dominated, it is discarded and the algorithm proceeds to the next iteration. Otherwise, l is used to filter $L(\vec{v}(l))$ and then added to $L(\vec{v}(l))$, where filtering means removing any existing labels in $L(\vec{v}(l))$ that are dominated by l. If l is complete, i.e. all target nodes have been visited (and all robots reach their goals if $v_g \neq null$), then a solution is found and post-optimized by the `PostOptimize`, which is explained later. After post-optimization, the algorithm breaks the inner loop, decrease ϵ, and continues the outer loop to search for better solutions until the time limit is reached. Otherwise, l is expanded by `SeqExpand` to generate successor labels. `SeqExpand` will select the robot that does not reach its goal yet and with the lowest cost in label l to get the next nodes of this robot. Each next node of this robot corresponds to a successor label. Then, for each successor label l', we discard it if it is infeasible according to `NotFeasible` or is dominated by existing labels in $L(\vec{v}(l'))$. If l' survives these checks, its f-value is computed by `Eval` as mentioned in Sec. IV-B. Then l' is added to L_O for future expansion. The focal list L_F is updated correspondingly so that L_F contains all labels l in L_O such that $f(l) \leq (1+\epsilon) f_{\min}$, with $f_{\min}$ being the minimum f-value of any label in L_O. The focal search terminates when the runtime limit is reached.

D. Constraint Check

The function `NotFeasible` in Alg. 1 checks each successor label l' during expansion to determine whether l' violates any constraints. This function considers AC, RC, and TW with waiting: For AC, each target node v can only be visited by robots in its admissible set $A(v)$. When a successor label assigns v to a robot $i \notin A(v)$, that label is discarded. For RC, for each robot i, the accumulated travel time $t^i(l')$ must satisfy $t^i(l') \leq T^i$. Any label violating this constraint is pruned. For TW, for a successor l that let robot i visit node v, the arrival time is computed same as in Sec. III: $t^i_{\text{arr}}(v) \leftarrow \max\{t_a(v),\ t^i(l) + c^i(v^i(l), v)\}$. If $t^i_{\text{arr}}(v) > t_b(v)$, the label is infeasible and is discarded.

TW constraints may render a label infeasible in the future. In particular, a label that is locally feasible at the current expansion may still have no feasible completion due to future time-window violations. To address the non-local infeasibility, we incorporate two additional pruning mechanisms that exploit structural properties of time-window feasibility:

(i) We adapt the classical *PostCheck* [20] idea to the RCH problem as follows. For a label l, we examine every unvisited node $u \in V \setminus B(l)$ and check whether there exists at least one robot that can reach u as its next visit without violating TW, AC or RC. If no such robot exists for some node u, then l cannot lead to a feasible solution and is thus pruned.

(ii) We also extend a pruning strategy called *LookAhead* [12] to the RCH problem as follows. The main idea is that, when generating a successor label l where robot i visits node v, if there exists another unvisited node u that robot i is allowed to visit such that visiting u first and then visiting v results in an arrival time at v that is no later than directly visiting v (after accounting for waiting), then the direct-to-v is redundant. In this case, the successor can be pruned because an alternative visitation order can produce a label that is no worse under the dominance relation while a strictly more nodes visited.

Algorithm 2: `PostOptimize`(l)

```
π ← BuildPaths(l)
if not Feasible(π; P) then
    return π
M ← Makespan(π)
π ← PathMerging(π, M; P)
π ← TwoOptFeasible(π; P)
return π
```

E. Post-Optimization

As shown in Alg. 2, the function `PostOptimize` in Alg. 1 is called after a feasible solution is found. It runs a local refinement to further reduce the objective value and tighten the incumbent upper bound. All refinement are validated under AC, RC and TW constraints. The refinement uses 2-opt [11] as the backbone local operator, where two edges in a path are perturbed by reversing a segment, and the resulting path is accepted only if it remains feasible and improves the objective. Beyond single-path 2-opt refinement, the post-optimization procedure also performs load balancing across robots. Robots are first partitioned into groups according to their types. Within each group, we iteratively transfer nodes from the longest path to the shortest path to reduce the inner-group makespan, followed by local 2-opt refinement. We further apply inter-group task transfer, where nodes on the globally longest path are selectively reassigned to robots in other groups with smaller inner-group makespan. An additional inner-group 2-opt refinement is then executed for further improvement.

Besides, due to the TW, some robots may wait long time to service a node. We introduce a path merge procedure to selectively merge shorter paths to other paths with long wait time without increasing the current makespan, thereby reducing redundant waiting. Specifically, the `PathMerging` loops over all robots' paths and all nodes in each path and seeks to insert a node that is visited by robot i to robot j's path if j's path contains long wait time, when such insertion does not increase the makespan. By doing so, a solution never increases its makespan, and can decrease its sum-of-costs. Although the stated RCH problem in Sec. III considers only minimizing the makespan, our algorithm seeks to also minimize the sum-of-costs after minimizing the makespan, which can be beneficial and useful in practice.

F. Relation to HEHA and Key Differences

1) Relation to HEHA and Algorithmic Extensions: Our PyRCH is based on the global planning algorithm in HEHA [7], which is an anytime focal-search algorithm for

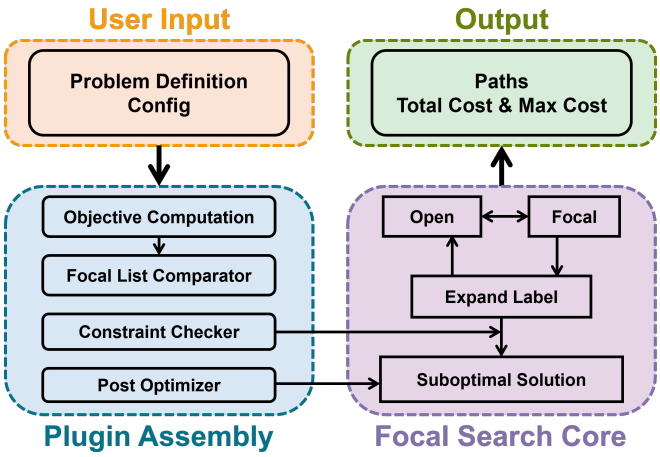


Fig. 2: Software architecture of our modular solver.

mTSP with AC. The major difference are the following. First, while HEHA considers only AC, PyRCH further considers RC and TW, which leads to more pruning rules in procedure `NotFeasible` on Line 16 in Alg. 1. Second, when computing the heuristics, PyRCH further considers TW when computing the MST as presented in Sec. IV-B. Third, for the post-optimization, PyRCH includes an additional path merging procedure as presented in Sec. IV-E.

2) Theoretical Guarantees: Our algorithm is complete and $(1+\epsilon)$-bounded sub-optimal. The analysis is similar to that in [7] for the following reasons. First, the new pruning rules only discard partial solutions that violate the constraint which never eliminate any feasible solution. Second, the post-optimization only refine a solution that is already found. Finally, the new component here is the TW-aware MST heuristic, which is still admissible for the following reasons. Intuitively, consider any feasible solution led by a label l. All unvisited nodes must be connected to the current partial solution through feasible transitions that respect time windows. When constructing the MST, we remove only edges that cannot be traveled by any robot in any direction, so no feasible solution uses on a discarded edge. For the remaining edges, the effective cost is lower-bounded by the minimum travel time with possible waiting time among all robots. Therefore, every feasible solution induces a spanning tree over the unvisited nodes whose total cost is no smaller than the constructed MST. $h(l)$ thus never overestimates the remaining cost-to-go, and the MST heuristic is admissible.

V. Solver Framework

A. Modular Software with Plugins

We implement PyRCH as a modular C++ solver with Python API for easy usage (Fig. 2). The core anytime focal-search procedure is fixed (Sec. IV-C), while problem-dependent part is encapsulated behind a set of interfaces. This separation allows adapting the solver to different objectives and constraint sets with minimal code changes in the core algorithm implementation.

The current package consists of the following interchangeable parts: (1) *Objective computation interface* evaluates each label l, enabling different optimization goals and heuristic designs; (2) *Focal prioritization interface* defines the ordering of labels within the focal list, allowing task-dependent search guidance; (3) *Constraint-checking interface* performs feasibility tests during expansion, including AC, RC, and TW constraints; (4) *Post-optimization interface* refines solutions to tighten the incumbent upper bound in anytime search.

In our implementation, these interfaces are realized via a lightweight plugin-style mechanism. So adding a new possible objective, constraint, or post optimization policy in the future only requires implementing the corresponding interface, without modifying the core search.

B. Python API

The Python package is released as `PyRCH` and can be installed via `pip install PyRCH`. The main `Planner` interface allows users to define nodes, robots, travel costs, AC, TW, RC, and solver options. It supports both incremental problem construction, which is useful when embedded in online robotic systems, and structured problem loading from benchmark files. Examples are provided within the package documentation.

VI. Experimental Results

A. Benchmark Construction

1) Maps and AC: Due to the lack of dataset with AC, we generate a benchmark dataset in a similar way to [7], with not only AC, but also RC and TW, based on three grid maps (Fig. 4) *battleground* (Map 1), *bootybay* (Map 2) and *divideandconquer* (Map 3), where each map is a 2D grid with heterogeneous terrain types, including ground, forest, swamp, water, and obstacle. We consider two types of robots, ground vehicle (GV) and aerial vehicle (AV). To generate an instance, we sample a few nodes from these grids maps to be visited by the robots, and generate a complete graph, where the edge costs are the shortest path cost between these sampled nodes using A* search in the grid map. These sampled nodes may locate on different terrains, and thus lead to AC. We classified the nodes to be visited by the robots into two classes: (i) *Normal nodes*, which are sampled from *ground* and *forest* grids so that both GVs and AVs can visit, (ii) *AV-only nodes*, which are sampled from *swamp* and *water* grids that can only be visited by AVs. The ratio of normal nodes and AV-only nodes is 2:1. There are four different settings: *Setting A* has 60 nodes, 3 GVs and 3 AVs; *Setting B* has 150 nodes, 10 GVs and 10 AVs; *Setting C* has 100 nodes, 2 GVs and 3 AVs; *Setting D* has 100 nodes, 3 GVs and 2 AVs. All settings are repeated 30 times, with nodes randomly sampled each time.

2) RC and TW Generation: To generate RC and TW constraints while ensuring feasibility, we first compute a feasible solution for each instance using a simple greedy algorithm. The time budget T^i for each robot i is then set to 1.2 times of $T_m^{\text{type(i)}}$, which is the max path length among all robots of that type. This construction guaranties feasibility

```python
import pyrch

planner = pyrch.Planner()

# Add nodes
planner.add_depot(0, x=0.0, y=0.0, z=0.0)
planner.add_target(1, x=1.0, y=2.0, z=0.0)
planner.add_target(2, x=3.0, y=1.0, z=0.0)

# Add agents with time-budget constraints
planner.add_agent(agent_id=0, agent_type="AV",
                  start_node=0, end_node=0,
                  time_limit=6.0)
planner.add_agent(agent_id=1, agent_type="GV",
                  start_node=0, end_node=0,
                  time_limit=9.0)

# Add cost matrices (per agent type)
planner.set_cost_matrix("AV", [
    [0.0, 2.24, 3.16],
    [2.24, 0.0, 2.24],
    [3.16, 2.24, 0.0],
])

planner.set_cost_matrix("GV", [
    [0.0, 1.80, 4.20],
    [1.80, 0.0, 2.90],
    [4.20, 2.90, 0.0],
])

# Define constraints (optional)
planner.add_assignment(1, ["AV", "GV"])
planner.add_assignment(2, ["GV"])
planner.add_time_window(1, start=0.0, end=3.0)
planner.add_time_window(2, start=0.0, end=6.0)

# Set objective and solver settings
planner.set_options(objective="min_max",
                    return_to_end=True,
                    time_limit=5.0)

result = planner.solve()
```

Fig. 3: Example usage of the RCH planner API.

while avoiding too loose RC. To generate TW, for each node i, let t_i denote the arrival time of the robot visiting node i in the greedy solution. We then build a symmetric time window centered at t_i: $[t_i - w_i,\ t_i + w_i]$, where $w_i = 0.1 \cdot T_m^{\text{type(i)}}$. This construction ensures that the greedy solution remains feasible while providing possibility for alternative solutions.

B. Baselines

We compare our solver with three baselines (B_{PyVRP}, $B_{\text{OR-Tools}}$, $B_{\text{OR-Tools*}}$) representing state-of-the-art routing solvers. B_{PyVRP} uses PyVRP [5], which optimizes the min-sum objective.[2] $B_{\text{OR-Tools}}$ uses Google OR-Tools [10] with default parameters, which optimizes the min-sum. $B_{\text{OR-Tools*}}$ is a fine tuned version of $B_{\text{OR-Tools}}$, including an augmented objective that encourages balanced joint path and smaller makespan. Specifically, we add a penalty term proportional to the difference between the longest and shortest path lengths: Objective $= \sum_{i \in I} L_i + 100 \cdot (\max_{i \in I} L_i - \min_{i \in I} L_i)$, where L_r denotes the path length of robot i.

[2]We communicate with the developer of PyVRP and learned that its current version can only optimize min-sum objective.

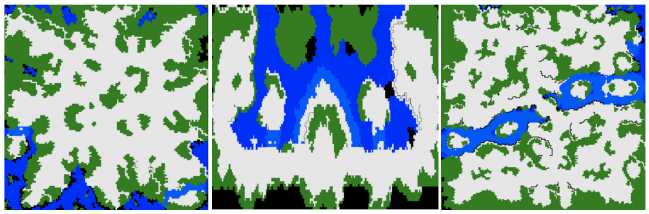


Fig. 4: Three grid maps of size 512×512 from a public dataset are used in our experiments to generate instances.

This penalty encourages balanced workloads among robots and leads to smaller makespan.

Note that neither PyVRP nor OR-Tools provides an explicit way to enforce AC. We implement AC indirectly by modifying the distance matrices (i.e., edge costs) for each robot: for each robot, edges connecting the robot to nodes that it is not allowed to visit are assigned a very large cost to avoid being included in the solution. Although this approach enforces assignment constraints in most cases, both PyVRP and OR-Tools may occasionally return infeasible solutions that violate AC. Whenever such violations occur, the solution is considered infeasible and excluded from the results. In our experiments, the termination condition for all solvers is set to 60 seconds time limit.

C. Experiments

We test three groups of instances: AC, AC+RC, AC+TW. When a solver returns a solution that violates AC, RC, or TW, the run is counted as a failure and excluded from the makespan and sum of costs comparisons.

1) AC: In the problems with AC only, all solvers achieve a 100% success rate. As shown in Fig. 5, our solver consistently achieves a lower makespan than all baselines. B_{PyVRP} and $B_{\text{OR-Tools}}$ generally achieve a lower sum of costs than ours, and $B_{\text{OR-Tools*}}$ (OR-Tools with makespan penalty) achieves a lower sum of costs in Setting B. The trade-off between makespan and sum of costs is expected, as our solver is designed for min-max as opposed to min-sum.

2) AC+RC: As shown in Fig. 6, our solver consistently achieves higher success rates than B_{PyVRP} and $B_{\text{OR-Tools}}$ on Setting A and also has almost the same success rate as B_{PyVRP} on Setting B. $B_{\text{OR-Tools}}$ and $B_{\text{OR-Tools*}}$'s success rates are unstable across different maps with Setting B. Our solver also consistently achieves lower makespan than all baselines. These results show that our solver effectively handles RC while maintaining both feasibility and solution quality.

3) AC+TW: We find that problems with TW are sensitive to the graph and the number of robots, and we thus test the solvers on all four settings A, B, C, D. As $B_{\text{OR-Tools*}}$ always performs better on makespan and has the same success rate compared with $B_{\text{OR-Tools}}$, we only show the results of $B_{\text{OR-Tools*}}$ here.

As shown in Fig. 8(a), our solver achieves success rates close to 100% for instances with Setting A on all maps. When the number of nodes increases to 150 in (b), the

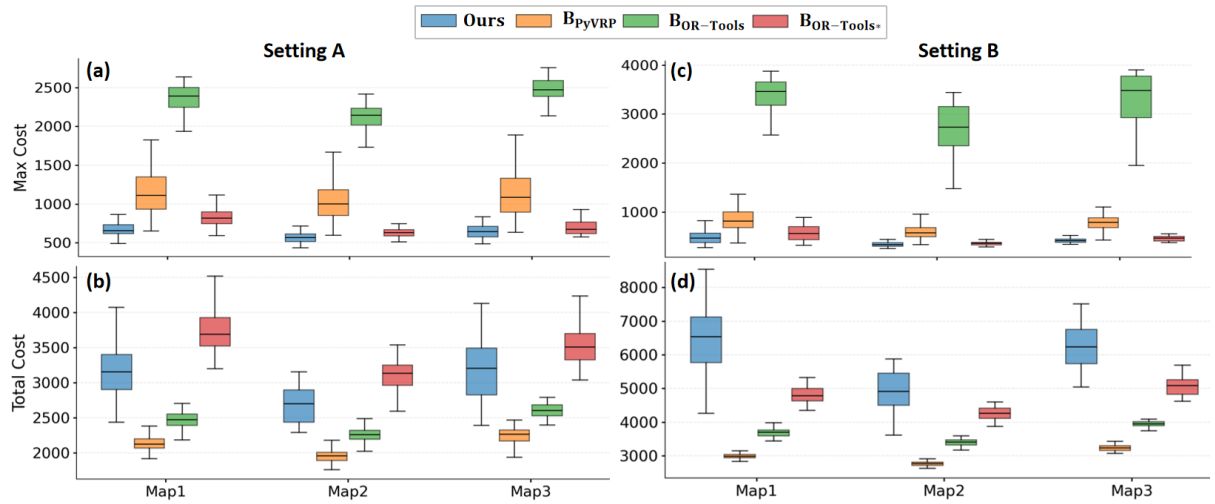


Fig. 5: Makespan and sum of costs for AC problems under Setting A (shown in (a),(b)) and Setting B (shown in (c),(d)).

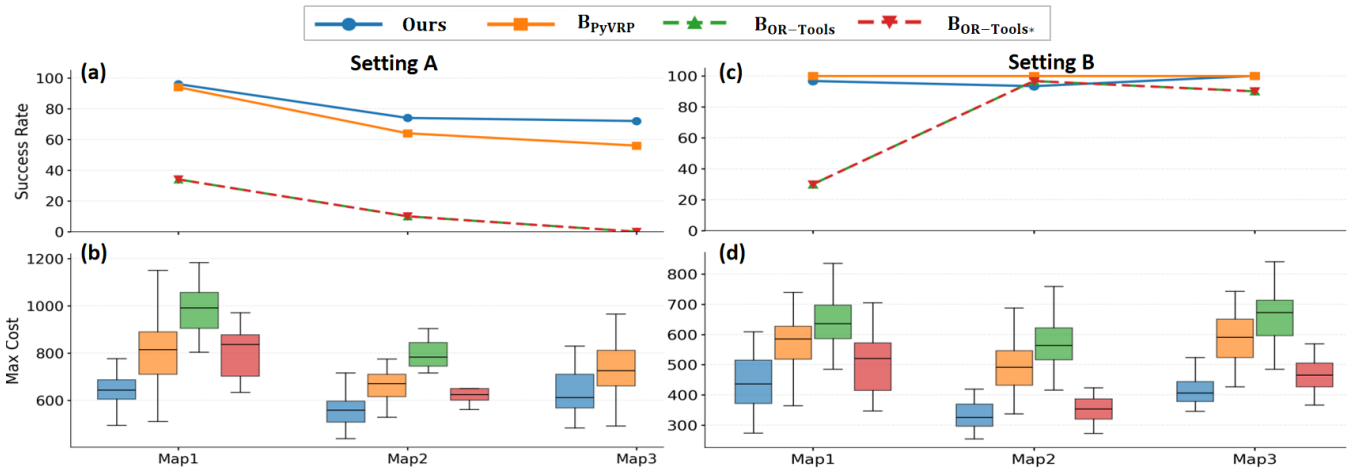


Fig. 6: Success rate and makespan for AC+RC problems under Setting A (shown in (a,b)) and Setting B (shown in (c,d)).

success rates drop noticeably, indicating that satisfying time-window constraints becomes more challenging. For instances with 100 nodes (c,d), the success rates exhibit only a moderate decline.

To compare solution quality, we use the solution by B_{PyVRP} as a reference to evaluate our solver and $B_{\text{OR-Tools*}}$: we report the makespan gap of our solver and $B_{\text{OR-Tools*}}$'s solution relative to B_{PyVRP}. As shown in Fig. 8, $B_{\text{OR-Tools*}}$ consistently produces solutions with substantially larger makespan than B_{PyVRP}. In contrast, our solver achieves solutions that are consistently close to B_{PyVRP}, with the makespan sometimes slightly higher and sometimes slightly lower than it. These results indicate that our solver achieves solution quality comparable to B_{PyVRP} under AC+TW.

4) Solution Visualization: To better understand the behavior of different solvers, we visualize representative solutions of Setting A on Map 1 under AC and RC in Fig. 1. Our solver produces well-balanced joint paths across robots, resulting in similar tour lengths and reduced makespan. In contrast, both B_{PyVRP} and $B_{\text{OR-Tools}}$ tend to assign tasks to a smaller subset of robots. $B_{\text{OR-Tools*}}$ partially alleviates this issue by encouraging more balanced assignments. However, its solution still has a larger makespan than ours.

The imbalance is further confirmed by the resource usage per robot shown in Fig. 1 (e). B_{PyVRP} and $B_{\text{OR-Tools}}$ exhibit highly uneven resource consumption among the robots. In contrast, our solver maintains more balanced resource usage.

5) Anytime Performance: Finally, we track the best makespan found over time by the solvers. Representative convergence curves of problems with AC or AC+RC in Setting A are shown in Fig. 7. Our solver quickly finds high-quality solutions (e.g. $< 0.1s$), while being able to further improve the solution a bit as time increases. In contrast, although B_{PyVRP} and $B_{\text{OR-Tools*}}$ can reduce the makespan gradually, their initial and final makespan are both higher than ours on average.

VII. CONCLUSION

This paper proposes an algorithm for min-max routing problems with robots under assignment constraints (AC), resource constraints (RC), and time window constraints (TW).

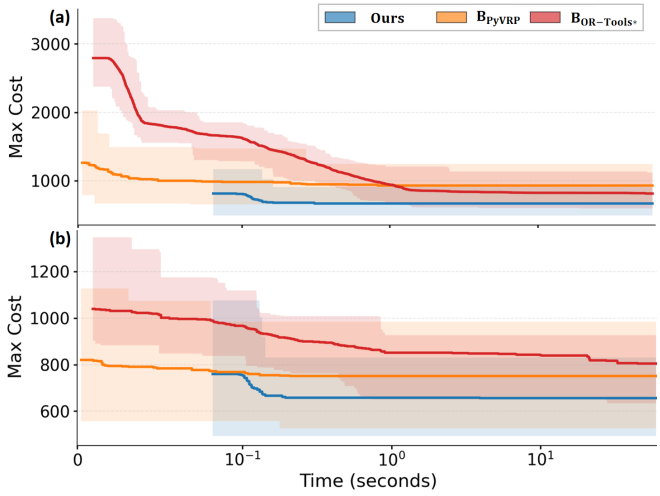


Fig. 7: The best makespan found over time of the solvers under Setting A on Map 1 for (a) AC and (b) AC+RC problems. Solid lines indicate the average across instances, and shaded regions denote the variance.

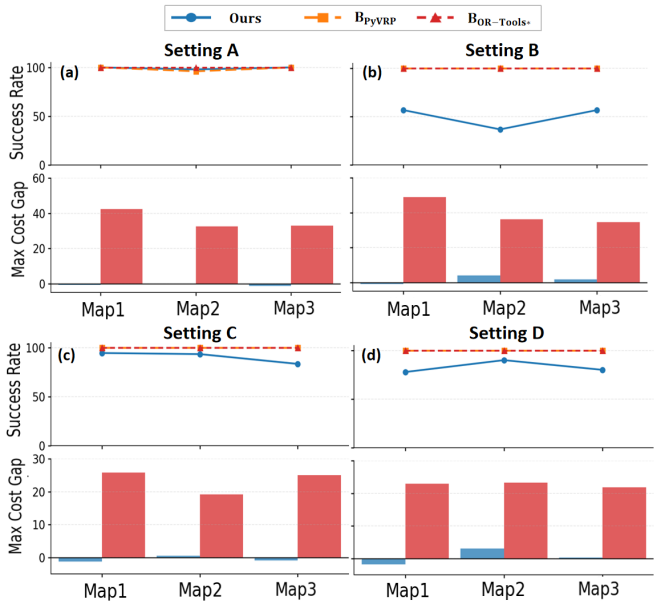


Fig. 8: Success rate (%) and makespan gap (%, relative to B_{PyVRP}) for AC+TW problems under Settings A, B, C and D on three maps.

The algorithm builds on an anytime focal-search framework with constraint-aware pruning, MST heuristic and post-optimization. In addition, we implement it as a modular C++ solver with a lightweight Python API. Experimental results show that the solver consistently achieves lower makespan than existing routing solvers such as PyVRP and OR-Tools across a range of test settings with anytime behavior.

Future work includes improving the solver's generality by considering more constraints and objectives. We also plan to develop task-specific focal prioritization strategies for different routing problems, allowing the focal search to better exploit problem structures and improve search efficiency.

ACKNOWLEDGEMENTS

The authors sincerely thank Jingfan Tang, an undergraduate student at Global College, Shanghai Jiao Tong University, for helpful comments and proofreading of the manuscript.

REFERENCES

[1] D. L. Applegate, R. E. Bixby, V. Chvatal, and W. J. Cook, *The Traveling Salesman Problem: A Computational Study (Princeton Series in Applied Mathematics)*. Princeton, NJ, USA: Princeton University Press, 2007.

[2] Y. Yang, Z. Liu, and W. Yu, "Approximation algorithms for the k-depots hamiltonian path problem," *Optimization Letters*, vol. 16, no. 4, pp. 1215–1234, 2022.

[3] R. Doshi, S. Yadlapalli, S. Rathinam, and S. Darbha, "Approximation algorithms and heuristics for a 2-depot, heterogeneous hamiltonian path problem," *International Journal of Robust and Nonlinear Control*, vol. 21, no. 12, pp. 1434–1451, 2011.

[4] K. Sundar and S. Rathinam, "Algorithms for heterogeneous, multiple depot, multiple unmanned vehicle path planning problems," *Journal of Intelligent & Robotic Systems*, vol. 88, pp. 513–526, 2017.

[5] N. A. Wouda, L. Lan, and W. Kool, "PyVRP: a high-performance VRP solver package," *INFORMS Journal on Computing*, vol. 36, no. 4, pp. 943–955, 2024.

[6] C. Cao, H. Zhu, Z. Ren, H. Choset, and J. Zhang, "Representation granularity enables time-efficient autonomous exploration in large, complex worlds," *Science Robotics*, vol. 8, no. 80, p. eadf0970, 2023.

[7] L. Yang, Y. Wang, J. Tang, Y. Lv, S. Zhao, C. Cao, and Z. Ren, "Heha: Hierarchical planning for heterogeneous multi-robot exploration of unknown environments," 2025.

[8] Z. Ren, S. Rathinam, and H. Choset, "CBSS: A new approach for multiagent combinatorial path finding," *IEEE Transactions on Robotics*, vol. 39, no. 4, pp. 2669–2683, 2023.

[9] X. Wu and Z. Ren, "Multi-agent combinatorial path finding for tractor-trailers in occupancy grids," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 14 124–14 131.

[10] V. Furnon and L. Perron, "Or-tools routing library," Google.

[11] K. Helsgaun, "General k-opt submoves for the lin–kernighan tsp heuristic," *Mathematical Programming Computation*, vol. 1, no. 2, pp. 119–163, 2009.

[12] S. Zhao, Y. Wu, and Z. Ren, "Bi-objective search for the traveling salesman problem with time windows and vacant penalties," in *Proceedings of the International Symposium on Combinatorial Search*, vol. 18, no. 1, July 2025, pp. 171–179.

[13] C. Cao, J. Xu, J. Zhang, H. Choset, and Z. Ren, "Heuristic search for the orienteering problem with time-varying reward," in *Proceedings of the International Symposium on Combinatorial Search*, vol. 17, 2024, pp. 11–19.

[14] P. Oberlin, S. Rathinam, and S. Darbha, "Today's traveling salesman problem," *IEEE robotics & automation magazine*, vol. 17, no. 4, pp. 70–77, 2010.

[15] A. G. Philip, Z. Ren, S. Rathinam, and H. Choset, "A mixed-integer conic program for the moving-target traveling salesman problem based on a graph of convex sets," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 8847–8853.

[16] E. Kwan, R. Qureshi, L. Fletcher, C. Laganier, V. Nockles, and R. Walters, "Onboard mission replanning for adaptive cooperative multi-robot systems," *IEEE Robotics and Automation Letters*, vol. 10, no. 12, p. 13225–13232, Dec. 2025.

[17] Y. Guo, Z. Ren, and C. Wang, "imtsp: Solving min-max multiple traveling salesman problem with imperative learning," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024, pp. 10 245–10 252.

[18] Z. Ren, S. Rathinam, and H. Choset, "Ms*: A new exact algorithm for multi-agent simultaneous multi-goal sequencing and path finding," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021, pp. 11 560–11 565.

[19] J. Pearl and J. H. Kim, "Studies in semi-admissible heuristics," *IEEE transactions on pattern analysis and machine intelligence*, no. 4, pp. 392–399, 1982.

[20] Y. Dumas, J. Desrosiers, E. Gelinas, and M. M. Solomon, "An optimal algorithm for the traveling salesman problem with time windows," *Operations research*, vol. 43, no. 2, pp. 367–371, 1995.